

SPECIFICATIONS TECHNIQUES

Architecture, algorithmes, formats d'echange et securite

Version : 1.0

IUT PTUT - Mars 2026

Migration bidirectionnelle WatchGuard Fireware <> pfSense

Document CONFIDENTIEL - Usage interne IUT PTUT

Sommaire

1. Stack technologique et justifications

Le choix de la stack technique de NETMORPH a été dicté par quatre critères : maturité de l'écosystème, richesse des bibliothèques XML disponibles en standard, facilité de packaging pour distribution Windows sans dépendances, et existence d'un framework UI moderne permettant le rendu HTML/CSS/JS natif dans une application desktop.

Technologie	Version	Rôle	Justification
Python 3.11	3.9+	Langage principal	Maturité, xml.etree stdlib, packaging PyInstaller, écosystème riche
PyQt6	6.4+	Framework UI desktop	UI native Windows/Linux, QWebEngineView, licence GPL/commercial
PyQt6-WebEngine	6.4+	Rendu HTML dans l'app	Chromium intégré, HTML5/CSS3/JS, communication IPC par URL scheme
xml.etree.ElementTree	stdlib	Parsing XML	Stdlib Python, léger, pas de dépendance externe, suffisant pour configs firewall
google-generativeai	0.3+	IA auto-réparation	Gemini 1.5 Flash : rapide, économique, excellent pour correction XML
python-dotenv	1.0+	Gestion secrets	Chargement .env sans hardcoding de la clé API dans le code
PyInstaller	5.x+	Packaging	Génère .exe autonome incluant Python runtime + Qt + Chromium (~300 Mo)
matplotlib	3.x+	Génération schémas	Diagrammes PNG pour la documentation technique

2. Architecture MVC — Vue détaillée

NETMORPH implémente rigoureusement le patron MVC avec une séparation stricte permettant de modifier chaque couche indépendamment. La Vue peut être entièrement redessinée sans toucher à la logique métier. Le Modèle peut être étendu (nouveaux formats) sans impacter l'interface utilisateur.

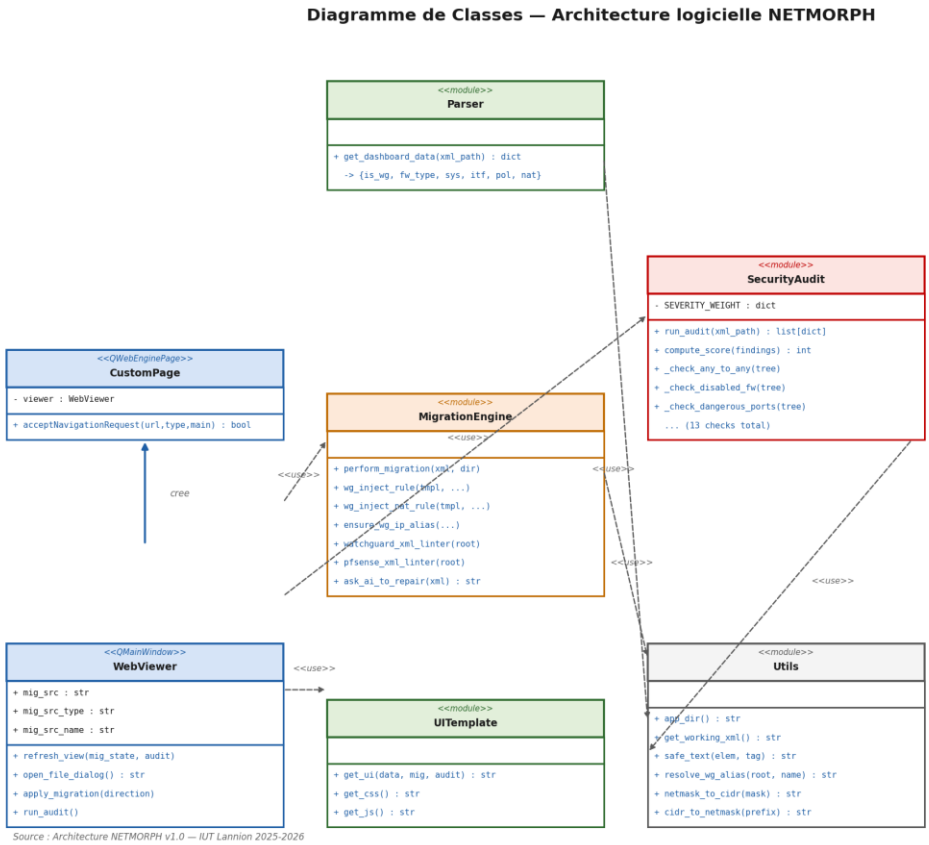


Figure 1 — Diagramme de classes NETMORPH : modules, attributs, méthodes et dépendances

2.1 Modèle — engine.py et parser.py

Le module parser.py est dédié à la lecture et à l'extraction d'informations depuis les fichiers XML. La fonction get_dashboard_data(xml_path) parse le fichier actif et retourne un dictionnaire Python structuré contenant six clés principales : is_wg (booléen de type), fw_type (label texte), sys (hostname et modèle), itf (liste des interfaces réseau), pol (liste des politiques de filtrage) et nat (liste des règles NAT). Cette structure est transmise à la Vue pour l'affichage du tableau de bord.

Le module engine.py contient toute la logique de transformation XML. Il expose quatre fonctions publiques : perform_migration(src_path, tgt_path) pour la migration complète, save_rule_xml(data) pour l'injection d'une règle manuelle, save_interface_xml(data) pour l'ajout d'une interface, et delete_rule_xml(idx) pour la suppression par index. Chaque écriture passe obligatoirement par les linters XML avant persistance.

2.2 Vue — ui_template.py

La Vue génère dynamiquement à chaque appel de refresh_view() le HTML, CSS et JavaScript complets injectés dans QWebEngineView. Elle reçoit en paramètre les données du parser, l'état du wizard de migration et les résultats de l'audit, mais ne contient aucune logique métier. Les fonctions get_ui(data, mig_state, audit_results), get_css() et get_js() produisent l'intégralité de l'interface.

L'interface est structurée en 4 onglets : Tableau de bord (topologie et règles), Éditeur rapide (formulaires d'ajout), Migrateur (wizard 2 étapes) et Audit de Sécurité (score + findings). Chaque action utilisateur est encodée comme un lien href='netmorph://action?params' ou un appel JavaScript window.location.href. Aucun formulaire HTML classique n'est utilisé.

2.3 Contrôleur — main.py

WebViewer hérite de QMainWindow et gère l'état de l'application (mig_src, mig_src_type, mig_src_name pour le wizard de migration). CustomPage hérite de QWebEnginePage et override acceptNavigationRequest() pour intercepter les URLs netmorph://. La méthode refresh_view() régénère le HTML complet depuis le parser et l'audit à chaque modification et l'injecte dans QWebEngineView via setHtml().

3. Protocole IPC — URL Scheme netmorph://

NETMORPH utilise un schéma d'URL propriétaire netmorph:// comme protocole de communication entre la vue HTML (côté navigateur) et le code Python (côté serveur). Ce mécanisme est plus simple et plus robuste que les alternatives classiques (WebSocket, JavaScript bridge, QWebChannel) tout en offrant une traçabilité parfaite des actions.

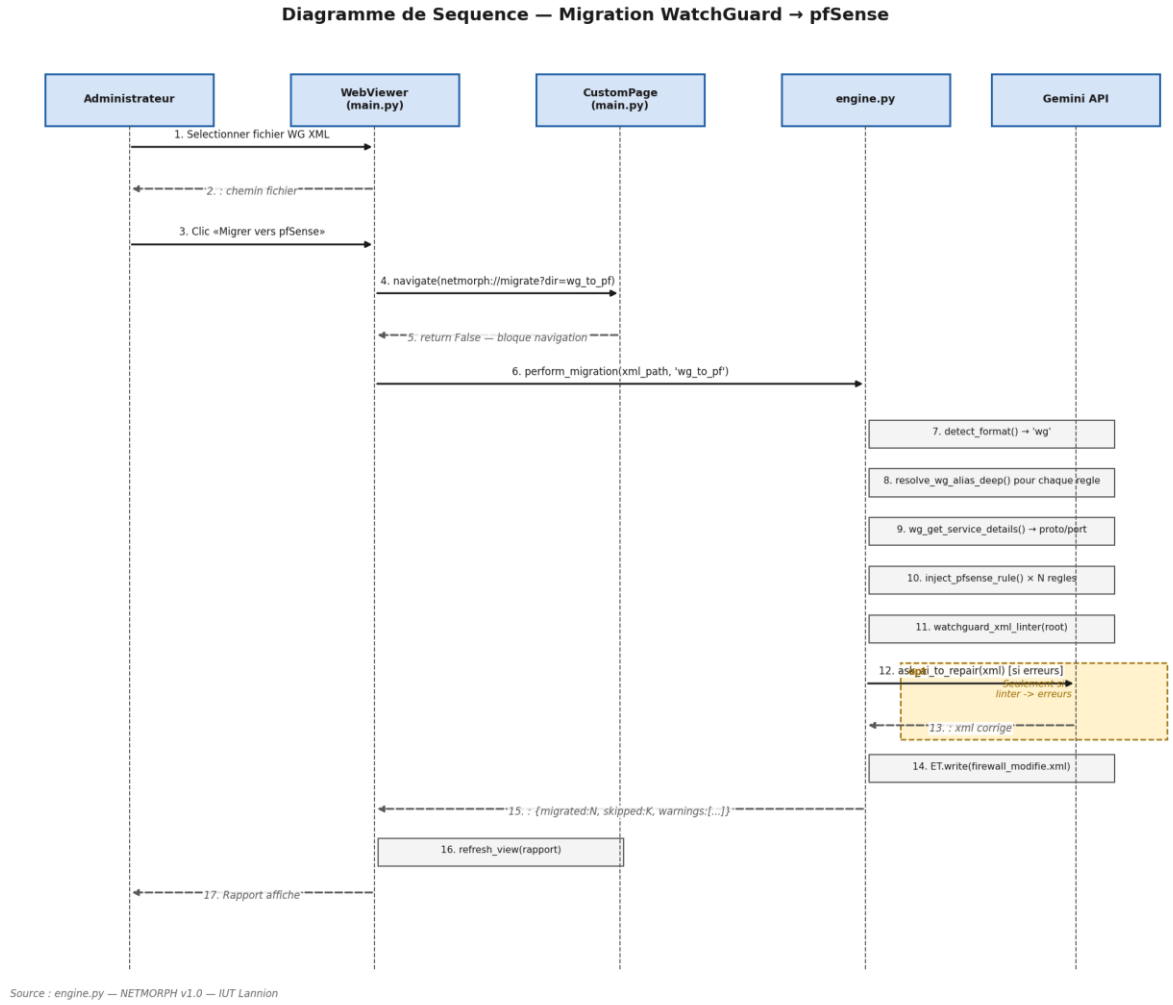


Figure 2 — Diagramme de séquence : migration WatchGuard → pfSense étape par étape

Le cycle complet d'une action utilisateur comprend 8 étapes : (1) l'utilisateur clique, (2) la vue génère une navigation netmorph://action?params, (3) Qt intercepte via acceptNavigationRequest(), (4) l'URL est parsée avec urlparse() et parse_qs(), (5) la méthode Python correspondante est appelée, (6) le Modèle effectue l'opération et met à jour firewall_modifie.xml, (7) refresh_view() régénère le HTML complet, (8) setHtml() injecte le nouveau rendu. Le return False final bloque la navigation effective — la page ne change jamais d'URL.

Action netmorph://	Paramètres QueryString	Traitement Python	Résultat
import	Aucun	QFileDialog + shutil.copy	firewall.xml mis à jour
export	Aucun	QFileDialog + shutil.copy	XML exporté vers chemin

			utilisateur
add_rule	name, action, service, from, to	save_rule_xml(data) + linter	Règle injectée dans firewall_modifie.xml
add_if	name, ip, type, if_dev	save_interface_xml(data)	Interface injectée dans XML
pick_source	Aucun	QFileDialog + détection root.tag	mig_src/type/name stockés
migrate	dest (WG ou pfSense)	perform_migration() + template	firewall_modifie.xml créé
reset_mig	Aucun	mig_src = mig_src_type = None	Wizard réinitialisé
delete_rule	index (int)	delete_rule_xml(idx)	Règle supprimée par index

4. Algorithmes de migration

4.1 Résolution récursive des alias WatchGuard

WatchGuard utilise un système d'alias en couches imbriquées. Un alias peut pointer vers un autre alias (type=2), une adresse directe (type=1) ou un address-group. La résolution récursive est indispensable pour obtenir les IPs réelles nécessaires à la création des règles pfSense. L'algorithme implémente un DFS (Depth-First Search) avec un ensemble 'visited' anti-cycle pour garantir la terminaison même en cas de références circulaires.

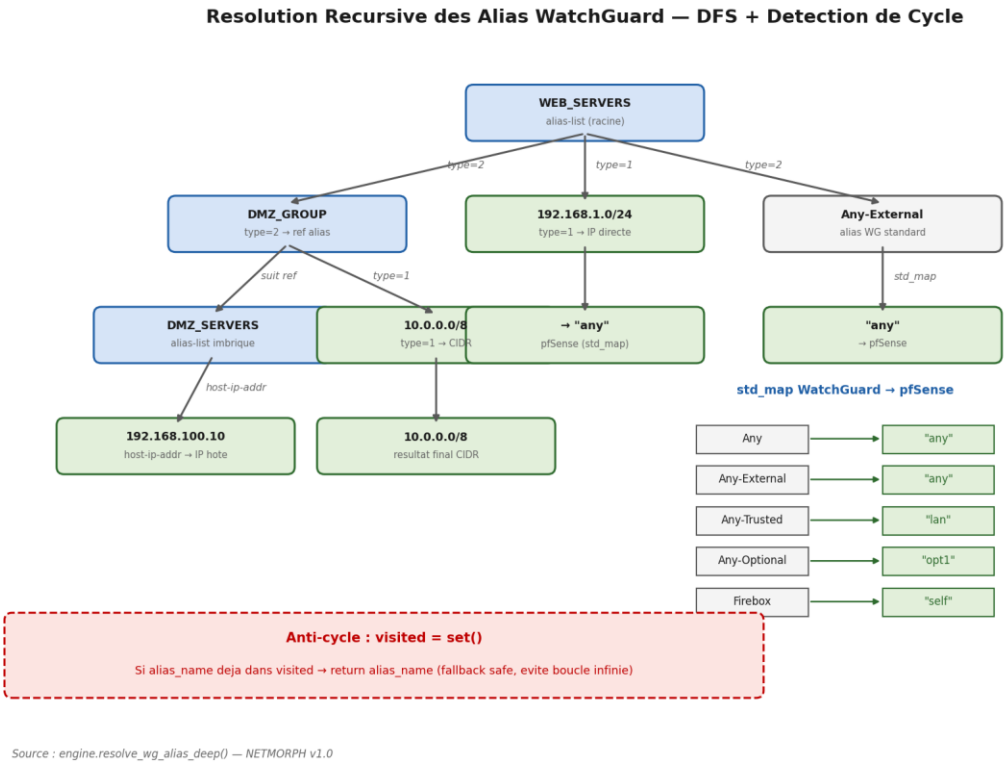


Figure 3 — Résolution récursive des alias WatchGuard : arbre DFS avec détection de cycle

L'algorithme traite en priorité les alias standards WatchGuard (Any → 'any', Any-External → 'any', Any-Trusted → 'lan', Firebox → 'wanip') avant de plonger dans les structures alias-list et address-group-list. En cas d'absence de résolution (alias introuvable), le nom de l'alias est retourné tel quel, ce qui constitue un fallback safe permettant à la migration de continuer sans crasher.

```
def resolve_wg_alias_deep(root, alias_name, _visited=None):
    if _visited is None: _visited = set()
    if alias_name in _visited: return alias_name   # Anti-cycle
    _visited.add(alias_name)
    std_map = {"Any":"any","Any-External":"any","Any-
Trusted":"lan","Firebox":"wanip"}
    if alias_name in std_map: return std_map[alias_name]
    # Parcours alias-list
    for al in root.findall(".//alias-list/alias"):
        if safe_text(al,"name") == alias_name:
            mem = al.find(".//alias-member-list/alias-member")
```

```
            if safe_text(mem,"type") == "2":   # ref vers autre alias
                return resolve_wg_alias_deep(root, safe_text(mem,"alias-name"),
_visited)
            if safe_text(mem,"type") == "1":   # adresse directe
                return resolve_wg_alias_deep(root, safe_text(mem,"address"),
_visited)
    # Fallback address-group-list
    for ag in root.findall(".//address-group-list/address-group"):
        if safe_text(ag,"name") == alias_name:
            return safe_text(ag,".//host-ip-addr") or safe_text(ag,".//ip-network-
addr")
    return alias_name   # fallback safe
```

4.2 Construction SNAT WatchGuard — 3 objets XML interdépendants

Un SNAT WatchGuard (Static NAT, type=7 = port forwarding) est une structure en 3 objets XML liés par convention de nommage. Cette architecture est une contrainte du moteur WatchGuard Fireware qui refuse les IPs brutes dans les champs de destination des policies liées à un NAT. NETMORPH crée automatiquement ces 3 objets lors de la migration des règles nat/rule pfSense.

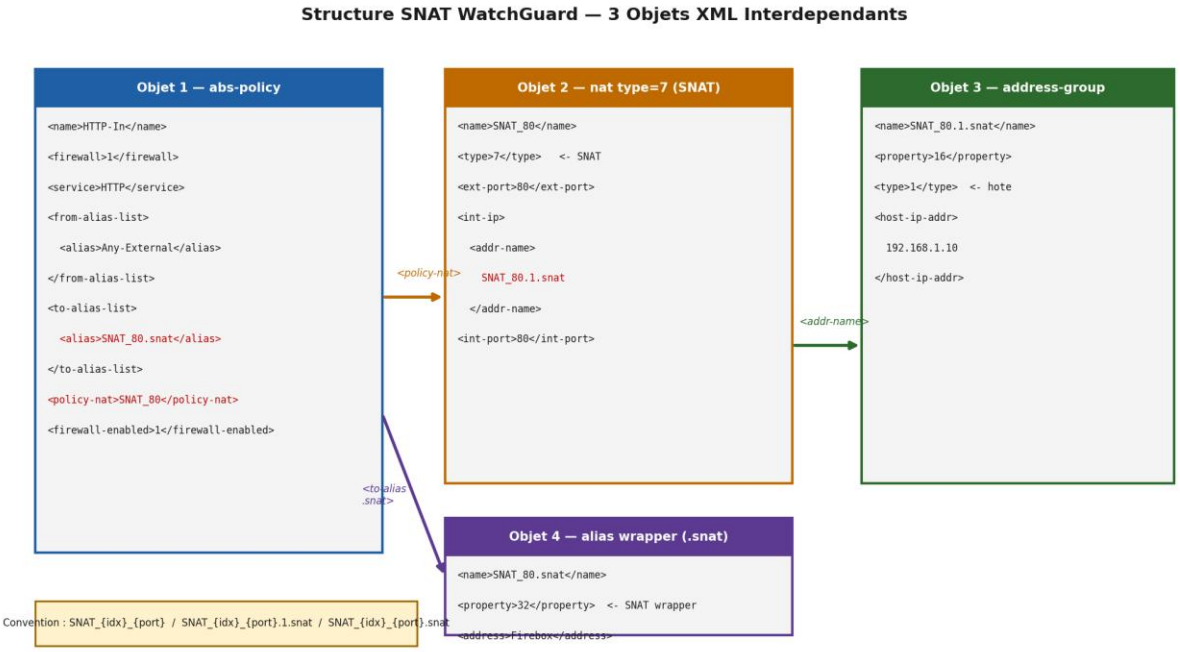


Figure 4 — Structure SNAT WatchGuard : 3 objets XML interdépendants et leurs liaisons

4.3 Validation XML et auto-réparation IA

Avant toute persistance dans firewall_modifie.xml, le XML généré passe par deux linters. Le linter WatchGuard détecte la présence de <source-port> et <source-port-enabled> dans les policies — ces balises sont rejetées par Fireware 12.x avec une erreur 400 lors de la restauration. Le linter pfSense vérifie la présence obligatoire de <ipprotocol> (inet ou inet6) dans chaque <rule> — son absence cause le silence silencieux de pfSense au boot (règle ignorée sans message d'erreur).

En cas d'échec du linter et si GEMINI_API_KEY est configurée, ask_ai_to_repair() envoie le XML en erreur à Gemini 1.5 Flash avec le message d'erreur précis. L'IA est guidée par un prompt système 'expert DevSecOps XML {OS}' et retourne le XML corrigé. Un post-traitement supprime les éventuelles balises markdown ```xml que l'IA peut ajouter.

```
Flux de validation/réparation :
1. Générer le XML cible (migration ou injection)
2. watchguard_xml_linter(xml_str) ou pfsense_xml_linter(xml_str)
3. Si (True, "OK")  → persistance directe dans firewall_modifie.xml
4. Si (False, err)  → ask_ai_to_repair(xml_str, err, target_os)
   4a. Si pas de clé API → persistance sans réparation (mode dégradé)
   4b. Si clé OK        → Gemini corrige → re-validation → persistance
```

5. Formats d'échange XML

5.1 WatchGuard Fireware 12.x

La structure XML WatchGuard est fondée sur un système de double-nœud pour les règles de filtrage : chaque règle visible dans la Web UI (abs-policy dans abs-policy-list) doit être accompagnée d'une règle moteur interne (policy dans policy-list) portant le même nom suffixé de '-00'. Sans ce double nœud, la règle est soit invisible, soit non fonctionnelle. Les règles système (property=32) sont gérées exclusivement par le firmware et doivent être préservées intactes lors des migrations.

```
<profile>
  <abs-policy-list>
    <abs-policy>
      <name>HTTP-In</name>
      <firewall>1</firewall>       <!-- 1=allow, 0=deny -->
      <service>HTTP</service>
      <property>0</property>       <!-- 0=user, 32=system (cache) -->
      <enabled>true</enabled>      <!-- OBLIGATOIRE -->
      <from-alias-list><alias>Any-External</alias></from-alias-list>
      <to-alias-list><alias>192.168.1.10</alias></to-alias-list>
      <policy-nat>SNAT_0_80</policy-nat>  <!-- optionnel: lien NAT -->
    </abs-policy>
  </abs-policy-list>
  <policy-list>
    <policy>                       <!-- noeud moteur interne lie -->
      <name>HTTP-In-00</name>      <!-- convention: abs-policy + "-00" -->
      <property>0</property>
      <enable>1</enable>           <!-- OBLIGATOIRE pour activer -->
    </policy>
  </policy-list>
</profile>
```

5.2 pfSense 2.x

Le format pfSense est structurellement plus simple : chaque règle de filtrage est un nœud <rule> direct dans <filter>. La balise <ipprotocol> est obligatoire (absence silencieusement ignorée par pfSense au boot). Les interfaces sont référencées par leur nom court (wan, lan, opt1). Les alias pfSense sont stockés dans <aliases> et peuvent être de type host, network ou port.

```
<pfsense>
  <filter>
    <rule>
      <type>pass</type>            <!-- pass ou block -->
      <interface>wan</interface>
      <protocol>tcp</protocol>
      <ipprotocol>inet</ipprotocol><!-- OBLIGATOIRE : inet/inet6/inet46 -->
      <source><any/></source>
      <destination>
        <address>192.168.1.10</address>
        <port>80</port>
      </destination>
      <descr>HTTP vers serveur web</descr>
      <log/>                       <!-- active le logging -->
    </rule>
  </filter>
  <nat>
    <rule>
```

```
      <interface>wan</interface>
      <protocol>tcp</protocol>
      <destination><port>80</port></destination>
      <target>192.168.1.10</target>
      <local-port>80</local-port>
    </rule>
  </nat>
</pfsense>
```

6. Sécurité et contraintes non-fonctionnelles

La sécurité est une préoccupation centrale dans la conception de NETMORPH, qui traite des fichiers de configuration firewall potentiellement sensibles contenant des informations sur l'architecture réseau interne de l'entreprise.

Mesure de sécurité	Description	Implémentation	Niveau
Isolation locale	Aucune transmission réseau des configs XML (sauf IA optionnel)	Design architectural	Critique
Validation à l'import	root.tag vérifié avant copie du fichier	main.py : import_file()	Haute
Validation avant écriture	Linters XML avant toute persistance	engine.py : linters	Haute
Secret isolation	.env gitignored, clé API absente par défaut	DevSecOps + .gitignore	Haute
Préservation original	firewall.xml jamais modifié, rollback possible	engine.py : get_working_xml()	Haute
Exception handling	try/except sur toutes opérations critiques	main.py : tous les handlers	Haute
Pas de réseau natif	Aucun port ouvert, aucune connexion sortante sauf IA	Architecture	Critique